

CYBR 437 Secure Coding

Winter 2026

Lab - 4

Release date: January 27, 2026

Due date: February 2, 2026, at 11:59 pm

Introduction

In this lab, you will translate assembly code into C, identify buffer overflow vulnerabilities in C code, and revise the code to remove the identified vulnerabilities.

Part 1 (50 points, 10 points per task):

In this section, you will analyze the following assembly code and write some equivalent C source code. In other words, you are translating the assembly code back to C files.

You will use variable names of your choice, and it is possible to have minor variations in the C code that could result in the same assembly code.

For each task, include your C code and detailed explanations on which assembly instructions correspond to which C statements in your code. You will not receive any credit if your submission includes only the code without appropriate explanations.

1.

(gdb) disass

Dump of assembler code for function **main**:

```
0x000055555555129 <+0>:      endbr64
0x00005555555512d <+4>:      push    %rbp
0x00005555555512e <+5>:      mov     %rsp,%rbp
=> 0x000055555555131 <+8>:      movl    $0x19,-0x4(%rbp)
0x000055555555138 <+15>:     movl    $0x1e,-0x8(%rbp)
0x00005555555513f <+22>:     movq    $0x2d,-0x10(%rbp)
0x000055555555147 <+30>:     mov     -0x4(%rbp),%edx
0x00005555555514a <+33>:     mov     -0x8(%rbp),%eax
0x00005555555514d <+36>:     add     %edx,%eax
0x00005555555514f <+38>:     cltq
0x000055555555151 <+40>:     sub     -0x10(%rbp),%rax
0x000055555555155 <+44>:     mov     %rax,-0x10(%rbp)
0x000055555555159 <+48>:     mov     $0x0,%eax
0x00005555555515e <+53>:     pop     %rbp
0x00005555555515f <+54>:     ret
```

End of assembler dump.

2.

(gdb) disass

Dump of assembler code for function `main`:

```
0x000055555555129 <+0>:      endbr64
0x00005555555512d <+4>:      push    %rbp
0x00005555555512e <+5>:      mov     %rsp,%rbp
=> 0x000055555555131 <+8>:      movl    $0x19,-0x4(%rbp)
0x000055555555138 <+15>:     movl    $0x1e,-0x8(%rbp)
0x00005555555513f <+22>:     mov     -0x4(%rbp),%eax
0x000055555555142 <+25>:     cmp     -0x8(%rbp),%eax
0x000055555555145 <+28>:     jle     0x55555555150 <main+39>
0x000055555555147 <+30>:     movl    $0x1,-0xc(%rbp)
0x00005555555514e <+37>:     jmp     0x55555555157 <main+46>
0x000055555555150 <+39>:     movl    $0x0,-0xc(%rbp)
0x000055555555157 <+46>:     mov     $0x0,%eax
0x00005555555515c <+51>:     pop     %rbp
0x00005555555515d <+52>:     ret
```

End of assembler dump.

3.

(gdb) disass

Dump of assembler code for function `main`:

```
0x000055555555129 <+0>:      endbr64
0x00005555555512d <+4>:      push    %rbp
0x00005555555512e <+5>:      mov     %rsp,%rbp
=> 0x000055555555131 <+8>:      movq    $0xa,-0x40(%rbp)
0x000055555555139 <+16>:     movq    $0x14,-0x38(%rbp)
0x000055555555141 <+24>:     movq    $0x1e,-0x30(%rbp)
0x000055555555149 <+32>:     movq    $0x28,-0x28(%rbp)
0x000055555555151 <+40>:     movq    $0x32,-0x20(%rbp)
0x000055555555159 <+48>:     movq    $0x0,-0x10(%rbp)
0x000055555555161 <+56>:     jmp     0x55555555175 <main+76>
0x000055555555163 <+58>:     mov     -0x10(%rbp),%rax
0x000055555555167 <+62>:     mov     -0x40(%rbp,%rax,8),%rax
0x00005555555516c <+67>:     add     %rax,-0x8(%rbp)
0x000055555555170 <+71>:     addq    $0x1,-0x10(%rbp)
0x000055555555175 <+76>:     cmpq    $0x4,-0x10(%rbp)
0x00005555555517a <+81>:     jle     0x55555555163 <main+58>
0x00005555555517c <+83>:     mov     $0x0,%eax
0x000055555555181 <+88>:     pop     %rbp
0x000055555555182 <+89>:     ret
```

End of assembler dump.

4.

(gdb) disass

Dump of assembler code for function **main**:

```
0x000055555555144 <+0>:      endbr64
0x000055555555148 <+4>:      push    %rbp
0x000055555555149 <+5>:      mov     %rsp,%rbp
=> 0x00005555555514c <+8>:      sub     $0x10,%rsp
0x000055555555150 <+12>:     movl    $0x5,-0x4(%rbp)
0x000055555555157 <+19>:     movl    $0x6,-0x8(%rbp)
0x00005555555515e <+26>:     mov     -0x8(%rbp),%edx
0x000055555555161 <+29>:     mov     -0x4(%rbp),%eax
0x000055555555164 <+32>:     mov     %edx,%esi
0x000055555555166 <+34>:     mov     %eax,%edi
0x000055555555168 <+36>:     call   0x55555555129 <fun>
0x00005555555516d <+41>:     mov     %eax,-0x4(%rbp)
0x000055555555170 <+44>:     mov     $0x0,%eax
0x000055555555175 <+49>:     leave
0x000055555555176 <+50>:     ret
```

End of assembler dump.

(gdb) disass fun

Dump of assembler code for function **fun**:

```
0x000055555555129 <+0>:      endbr64
0x00005555555512d <+4>:      push    %rbp
0x00005555555512e <+5>:      mov     %rsp,%rbp
0x000055555555131 <+8>:      mov     %edi,-0x4(%rbp)
0x000055555555134 <+11>:     mov     %esi,-0x8(%rbp)
0x000055555555137 <+14>:     mov     -0x4(%rbp),%edx
0x00005555555513a <+17>:     mov     -0x8(%rbp),%eax
0x00005555555513d <+20>:     add     %edx,%eax
0x00005555555513f <+22>:     sub     $0x1,%eax
0x000055555555142 <+25>:     pop     %rbp
0x000055555555143 <+26>:     ret
```

End of assembler dump.

5.

(gdb) disass

Dump of assembler code for function **main**:

```
0x000055555555195 <+0>:      endbr64
0x000055555555199 <+4>:      push    %rbp
0x00005555555519a <+5>:      mov     %rsp,%rbp
=> 0x00005555555519d <+8>:      mov     $0xa,%edx
0x0000555555551a2 <+13>:     mov     $0x14,%esi
0x0000555555551a7 <+18>:     mov     $0x1e,%edi
0x0000555555551ac <+23>:     call   0x5555555514b <fun1>
0x0000555555551b1 <+28>:     mov     $0x0,%eax
0x0000555555551b6 <+33>:     pop     %rbp
0x0000555555551b7 <+34>:     ret
```

End of assembler dump.

(gdb) disass fun1

Dump of assembler code for function fun1:

```
0x00005555555514b <+0>:      endbr64
0x00005555555514f <+4>:      push    %rbp
0x000055555555150 <+5>:      mov     %rsp,%rbp
0x000055555555153 <+8>:      sub     $0x28,%rsp
0x000055555555157 <+12>:     mov     %rdi,-0x18(%rbp)
0x00005555555515b <+16>:     mov     %rsi,-0x20(%rbp)
0x00005555555515f <+20>:     mov     %rdx,-0x28(%rbp)
0x000055555555163 <+24>:     mov     -0x18(%rbp),%rax
0x000055555555167 <+28>:     imul    -0x20(%rbp),%rax
0x00005555555516c <+33>:     mov     -0x28(%rbp),%rdx
0x000055555555170 <+37>:     imul    %rdx,%rax
0x000055555555174 <+41>:     mov     %rax,-0x8(%rbp)
0x000055555555178 <+45>:     mov     -0x8(%rbp),%rdx
0x00005555555517c <+49>:     mov     -0x18(%rbp),%rax
0x000055555555180 <+53>:     mov     %rdx,%rsi
0x000055555555183 <+56>:     mov     %rax,%rdi
0x000055555555186 <+59>:     call    0x55555555129 <fun2>
0x00005555555518b <+64>:     mov     %rax,-0x8(%rbp)
0x00005555555518f <+68>:     mov     -0x8(%rbp),%rax
0x000055555555193 <+72>:     leave
0x000055555555194 <+73>:     ret
```

End of assembler dump.

(gdb) disass fun2

Dump of assembler code for function fun2:

```
0x000055555555129 <+0>:      endbr64
0x00005555555512d <+4>:      push    %rbp
0x00005555555512e <+5>:      mov     %rsp,%rbp
0x000055555555131 <+8>:      mov     %rdi,-0x18(%rbp)
0x000055555555135 <+12>:     mov     %rsi,-0x20(%rbp)
0x000055555555139 <+16>:     mov     -0x18(%rbp),%rax
0x00005555555513d <+20>:     sub     -0x20(%rbp),%rax
0x000055555555141 <+24>:     mov     %rax,-0x8(%rbp)
0x000055555555145 <+28>:     mov     -0x8(%rbp),%rax
0x000055555555149 <+32>:     pop     %rbp
0x00005555555514a <+33>:     ret
```

End of assembler dump.

Part 2 (40 points, 10 points per task):

Each of the following C programs/code fragments has potential buffer overflow vulnerabilities. Identify the vulnerabilities and verify with appropriate test cases (*do not turn off the stack canary during compilation*) and the resulting output. Then, revise the code (*considering user inputs/source data invalid is not a valid revision*) to remove the vulnerabilities and explain how your revision eliminates the vulnerabilities. Finally, rerun the programs with the same test cases to verify the elimination of the vulnerabilities.

6.

```
#include <stdio.h>

int main() {
    char city[10];
    printf("Enter city: ");
    scanf("%s", city);
    printf("City: %s\n", city);
    return 0;
}
```
7.

```
# include <stdio.h>
int main()
{
    char buf[8];
    int i = 0;
    char c;
    while ((c = getchar()) != '\n') {
        buf[i++] = c;
    }
    buf[i] = '\0';
    printf("%s\n", buf);
    return 0;
}
```
8. A test case is not required for this task (the outputs are still needed).

```
#include <stdio.h>
int main()
{
    char buf[10];
    int i;

    for (i = 0; i <= 10; i++)
        buf[i] = 'A';

    printf("Done\n");
    return 0;
}
```

9. The following function copies `n` elements of the array `from` to the array `to`, starting at position `pos` in array `to`. Assume that the array `from` contains more than `n` elements. Write an appropriate main function to call `fun`.

```
int fun (double *to, double *from, int pos, int n)
{
    int k;
    for (k=0; k<n; k++)
    {
        to[pos] = from[k];
        pos++;
    }
    return pos;
}
```

Submission Instructions: Please submit a single PDF file in Canvas that includes your code, explanation, and screen capture(s) of the output, as applicable.